Sistemas Operativos II

Práctica 3 — Informe 2

Solución con mutex y variables de condición
del problema del productor-consumidor

Yago Regueira y Rodrigo Rivademar
Grado en Ingeniería Informática — USC

1. Introducción

En este informe se presenta la solución al problema de las carreras críticas identificadas en el Informe 1, utilizando mutex (pthread_mutex_t) y variables de condición (pthread_cond_t) en lugar de los semáforos con nombre empleados en la Práctica 2.

El objetivo es comprobar que esta solución elimina por completo las inconsistencias en la variable count, permite que el buffer se llene y vacíe correctamente según la velocidad de cada hilo la cual va variando, y garantiza que las sumas del productor y del consumidor coincidan al finalizar la ejecución.

2. Compilación y ejecución

El programa se compila con gcc:

```
gcc mutex_cons_prod.c -o mutex_cons_prod
```

Para ejecutarlo:

```
./mutex_cons_prod
```

Se utliza el mismo fichero que en el informe 1 de esta práctica.

3. Diseño de la solución

3.1. Mutex y variables de condición vs semáforos

En la Práctica 2 se utilizaban tres semáforos con nombre (`vacias`, `llenas`, `mutex`) creados con `sem_open` y sincronizando las esperas con `sem_wait/sem_post`. Estos semáforos se mantenían en /los archivos de nuestro Sistema Operativo por lo que era necesario llamar a `sem_unlink` al inicio del programa para limpiar posibles residuos de ejecuciones anteriores.

En esta solución se sustituyen por un `pthread_mutex_t` y dos `pthread_cond_t`, inicializados con `PTHREAD_MUTEX_INITIALIZER` y `PTHREAD_COND_INITIALIZER`. Al ser variables locales al proceso que viven en su espacio de memoria, no es necesario eliminarlas al principio, ya que desaparecen una vez finalizado el programa.

3.2. Patrón de sincronización

El patrón utilizado dentro de la región crítica es:

1. `pthread_mutex_lock`
2. `while(condición) pthread_cond_wait`
3. operar sobre el buffer

4. `pthread_cond_signal`

5. `pthread_mutex_unlock`.

Es crítico usar espera activa para que el hilo no se despierte hasta que se deje de cumplir la condición.

La variable de condición `no_lleno` la utiliza el productor para esperar cuando el buffer está lleno (`count == N`), y la señaliza el consumidor tras extraer un elemento. La variable `no_vacio` funciona de forma inversa: la espera el consumidor cuando `count == 0` y la señaliza el productor tras insertar.

3.3. Control de velocidades

Las llamadas a `sleep()` se realizan de la misma manera que en la Práctica 2 lo único que cambia es que en las iteraciones 60-79, ambos duermen un tiempo aleatorio entre 0 y 3 segundos.

3.4. Funciones principales

Las funciones `produce_item`, `insert_item`, `remove_item` y `consume_item` mantienen la misma lógica que en el Informe 1. La diferencia clave es que `insert_item` y `remove_item` ya no contienen `sleep()` entre la lectura y la escritura de `count`, ya que el mutex protege toda la operación. La actualización de count se realiza directamente con `count++` y `count--` dentro de la sección protegida.

4. Resultados

En esta sección se presentan las evidencias obtenidas de la ejecución del programa, organizadas según las tres fases de velocidad.

4.1. Iteraciones 0-29 (buffer se llena)

Durante las primeras 10 iteraciones del productor, el consumidor aún no ha extraído ningún elemento (está frenado por su `sleep(1)`). El productor llena el buffer de la posición 0 a la 9, momento en que count alcanza `N=10` y `pthread_cond_wait(&no_lleno, &mutex)` lo bloquea. A partir de ahí se establece un patrón característico: el consumidor extrae un elemento, señaliza `no_lleno`, el productor inserta uno nuevo y señaliza `no_vacio`, y el ciclo se repite con `count` oscilando entre 9 y 10.

```
[Iteración:  0]Productor: Introduciendo 63 en la posicion 0. Count: 0.
[Iteración:  0]Productor: Count actualizado a 1.
[Iteración:  1]Productor: Introduciendo 94 en la posicion 1. Count: 1.
[Iteración:  1]Productor: Count actualizado a 2.
[Iteración:  2]Productor: Introduciendo 50 en la posicion 2. Count: 2.
[Iteración:  2]Productor: Count actualizado a 3.
[Iteración:  3]Productor: Introduciendo 90 en la posicion 3. Count: 3.
[Iteración:  3]Productor: Count actualizado a 4.
[Iteración:  4]Productor: Introduciendo 11 en la posicion 4. Count: 4.
[Iteración:  4]Productor: Count actualizado a 5.
```

```
[Iteración:  9]Productor: Introduciendo 6 en la posicion 9. Count: 9.
[Iteración:  9]Productor: Count actualizado a 10.
[Iteración:  0]Consumidor: retiro 63 de la posicion 0. Count era: 10.
[Iteración:  0]Consumidor: Count actualizado a 9.
[Iteración: 10]Productor: Introduciendo 19 en la posicion 0. Count: 9.
[Iteración: 10]Productor: Count actualizado a 10.
```

Figura 1: Primeras iteraciones mostrando como se llena el buffer.

4.2. Iteraciones 30-59 (buffer se vacía)

Cuando el consumidor alcanza la iteración 30, deja de ejecutar `sleep(1)` y consume a velocidad máxima (Figura 2). El consumidor vacía rápidamente el buffer: `count` desciende desde 10 hasta 0. Una vez vacío, se establece el patrón inverso: cada vez que el productor inserta un entero (`count` sube a 1), el consumidor lo extrae inmediatamente (`count` vuelve a 0). El productor, aunque no está bloqueado,

```
[Iteración: 39]Productor: Introduciendo 25 en la posicion 9. Count: 9.
[Iteración: 39]Productor: Count actualizado a 10.
[Iteración: 30]Consumidor: retiro 31 de la posicion 0. Count era: 10.
[Iteración: 30]Consumidor: Count actualizado a 9.
[Iteración: 31]Consumidor: retiro 68 de la posicion 1. Count era: 9.
[Iteración: 31]Consumidor: Count actualizado a 8.
[Iteración: 32]Consumidor: retiro 64 de la posicion 2. Count era: 8.
[Iteración: 32]Consumidor: Count actualizado a 7.
[Iteración: 33]Consumidor: retiro 25 de la posicion 3. Count era: 7.
[Iteración: 33]Consumidor: Count actualizado a 6.
```

no tiene tiempo de introducir en el buffer al estar en fase lenta.

```
[Iteración: 39]Consumidor: retiro 25 de la posicion 9. Count era: 1.
[Iteración: 39]Consumidor: Count actualizado a 0.
[Iteración: 40]Productor: Introduciendo 9 en la posicion 0. Count: 0.
[Iteración: 40]Productor: Count actualizado a 1.
[Iteración: 40]Consumidor: retiro  9 de la posicion 0. Count era: 1.
[Iteración: 40]Consumidor: Count actualizado a 0.
```

Figura 2: Iteraciones mostrando como se vacía el buffer. Dispares al principio, pero se igualan.

4.3. Iteraciones 60-79 (aleatorio)

En las últimas 20 iteraciones, ambos hilos esperan tiempos aleatorios entre 0 y 3 segundos. Las iteraciones se intercalan de forma impredecible: en ocasiones el productor inserta varios elementos seguidos antes de que el consumidor actúe, y en otras ocurre lo contrario. (Figura 3)

```
[Iteración: 69]Productor: Introduciendo 86 en la posicion 9. Count: 0.
[Iteración: 69]Productor: Count actualizado a 1.
[Iteración: 70]Productor: Introduciendo 84 en la posicion 0. Count: 1.
[Iteración: 70]Productor: Count actualizado a 2.
[Iteración: 69]Consumidor: retiro 86 de la posicion 9. Count era: 2.
[Iteración: 69]Consumidor: Count actualizado a 1.
[Iteración: 70]Consumidor: retiro 84 de la posicion 0. Count era: 1.
[Iteración: 70]Consumidor: Count actualizado a 0.
[Iteración: 71]Productor: Introduciendo 54 en la posicion 1. Count: 0.
[Iteración: 71]Productor: Count actualizado a 1.
[Iteración: 71]Consumidor: retiro 54 de la posicion 1. Count era: 1.
[Iteración: 71]Consumidor: Count actualizado a 0.
```

Figura 3: Iteraciones aleatorias intercalando productor y consumidor

4.4. Sumas finales

Al finalizar las 80 iteraciones, ambos hilos imprimen su suma (Figura 4). Los resultados son idénticos, confirmando así que el mutex y las variables de condición eliminan por completo las inconsistencias observadas en el Informe 1.

```
RESULTADOS:
        Suma productor: 4008
        Suma consumidor: 4008
```

Figura 4: Resultados de la suma

5. Conclusiones

La solución con pthread_mutex_t y pthread_cond_t resuelve completamente el problema de las carreras críticas del Informe 1.

Las tres fases de velocidad demuestran que el mecanismo de sincronización funciona correctamente tanto cuando el buffer se llena, como cuando se vacía, como en condiciones aleatorias. Las sumas las cuales coinciden confirman que ningún dato se perdió ni se duplicó.